

Aprendizaje Automático

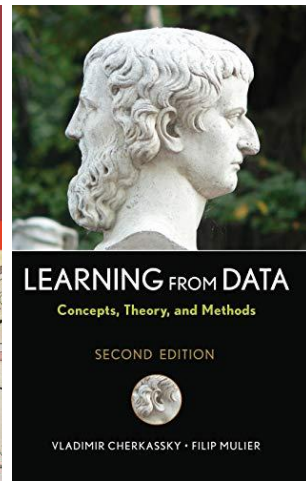
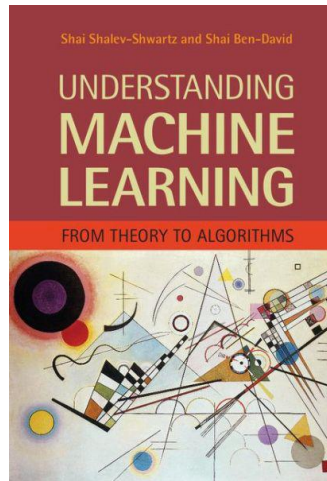
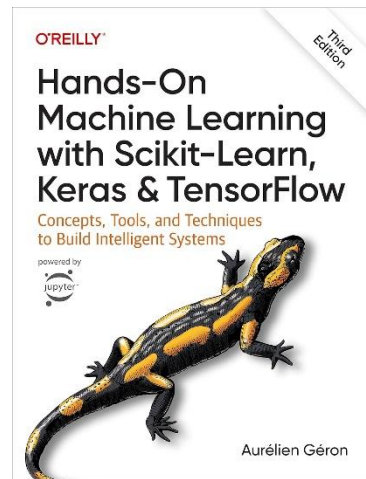
Curso 2025-2026

Tema 6. Aprendizaje Profundo en diferentes tipos de datos y problemas



**Redes Neuronales
Convolucionales.
Procesamiento de
imágenes**

Bibliografía



- 
- Tema 1: Conceptos básicos del Aprendizaje Automático
 - Tema 2: Aprendizaje Supervisado. Regresión y Clasificación. Modelos. Evaluación
 - Tema 3: Ingeniería de Características y Preprocesamiento de Datos. Calidad de Datos
 - Tema 4: Aprendizaje no Supervisado
 - Tema 5: Aprendizaje Profundo. Fundamentos
 - **Tema 6: Aprendizaje Profundo en diferentes tipos de datos y problemas**
 - Tema 7. Aprendizaje por refuerzo
 - Tema 8: Optimización en Aprendizaje Automático. Hiperparámetros, selección de modelos y regularización
 - Tema 9: Aspectos Avanzados, retos y desafíos del aprendizaje automático

Índice

Redes Neuronales Profundas

Redes neuronales convolucionales

¿Qué es una convolución?

Machine Learning vs Deep Learning

Autoencoders

Lectura recomendada

(buena parte de las siguientes *slides* están tomadas de estas fuentes)

- CS231n: Convolutional Neural Networks for Visual Recognition (ahora llamado Deep Learning for Computer Vision):

<https://cs231n.stanford.edu/slides/2023/>

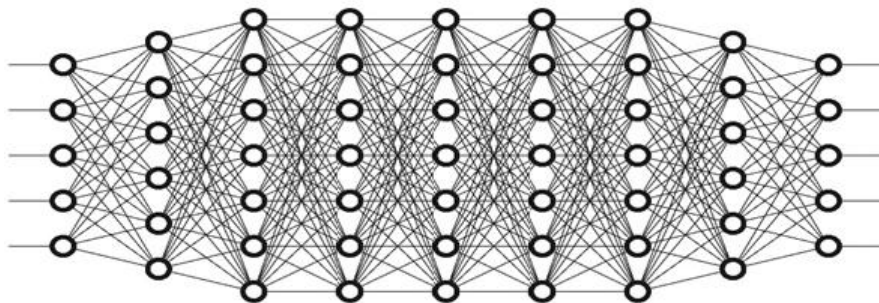
- En particular, la sección dedicada a Convolutional Neural Networks (ConvNets):

http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

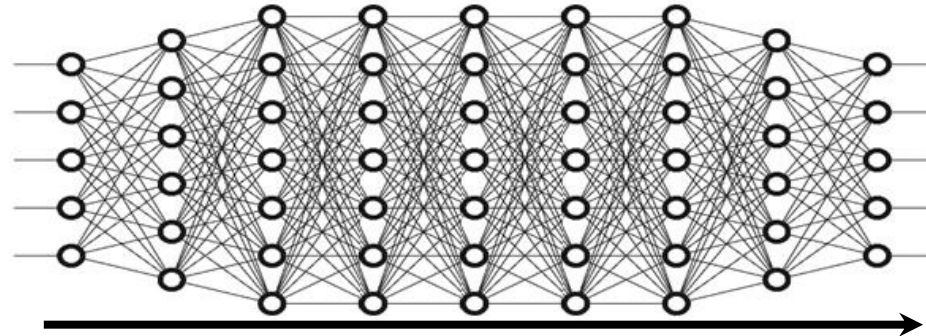
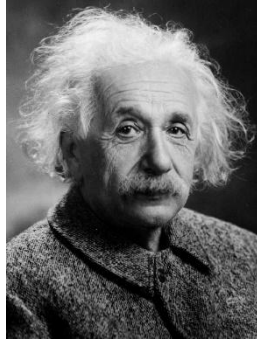
<https://cs231n.github.io/convolutional-networks/>

¿Qué son las redes neuronales profundas?

- Definición oficial
 - computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction (LeCun et al., 2015)
- A nivel práctico (y en general):
 - Redes neuronales con “muchas” capas ocultas
 - Y que, como consecuencia, pueden aproximar funciones más complejas (es decir, resolver problemas más complejos)

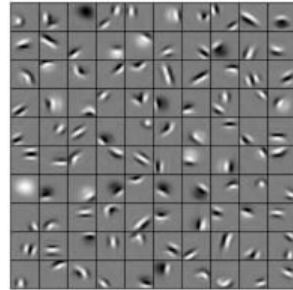


¿Qué son las redes neuronales profundas?

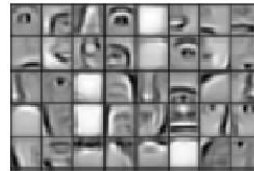


“Albert
Einstein”

Low-level
features



Mid-level
features



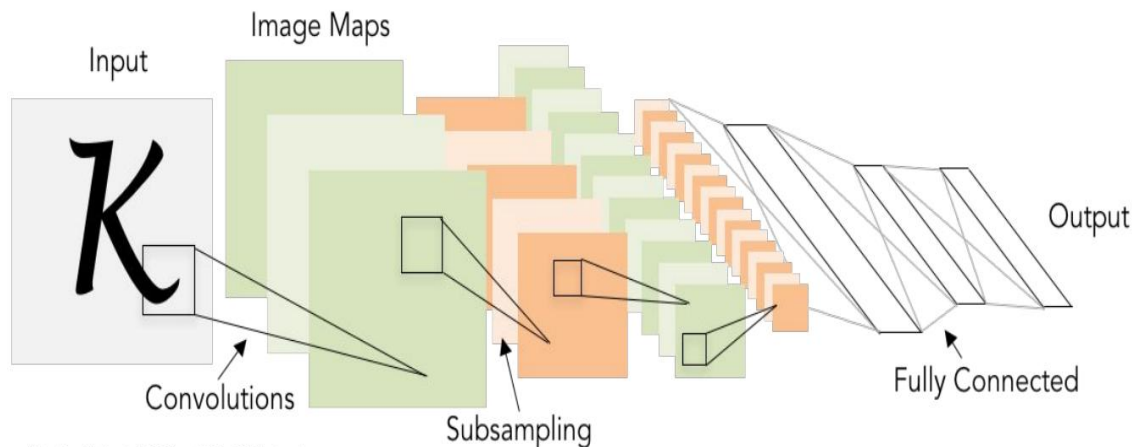
High-level
features



Aprenden representaciones jerárquicas de los datos de entrada
(múltiples niveles de abstracción)

Redes neuronales convolucionales

- ¿Qué es una red neuronal convolucional (ConvNet o CNN)?
 - Red neuronal con una convolución en, al menos, una de sus capas.
 - Se utiliza en problemas donde la información se presenta en formato de matriz/*grid* (por ejemplo, imágenes)
- Arquitectura típica de una ConvNet:



¿Qué es una convolución?

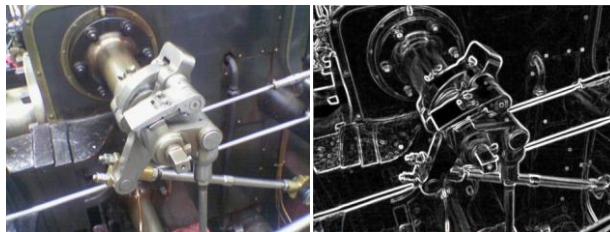
- **Operación local lineal** con una máscara (también llamado filtro o *kernel*).
 - Los coeficientes/valores de la máscara/filtro determinan la operación realizada.
 - Filtrado Gaussiano (elimina altas frecuencias → suavizado de imágenes)
- Sobel filter (elimina bajas frecuencias → realce de bordes)

$$\frac{1}{273}$$

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1



$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

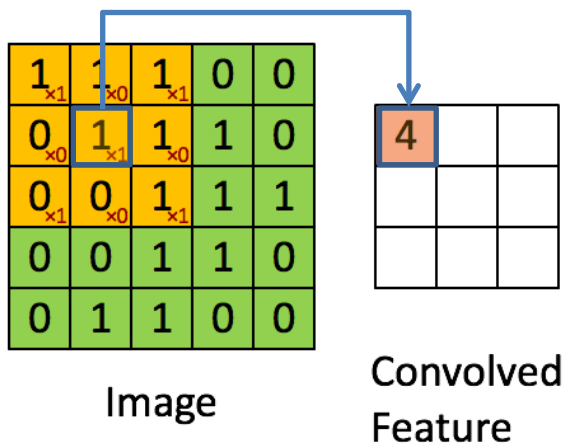


¿Qué es una convolución?

- Operación local lineal con una máscara.

Los números **rojos** representan los valores/coeficientes del filtro/máscara.

El filtro se multiplica elemento por elemento con la imagen, se suman los productos y se sustituye la posición central del filtro en la imagen.



1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

=

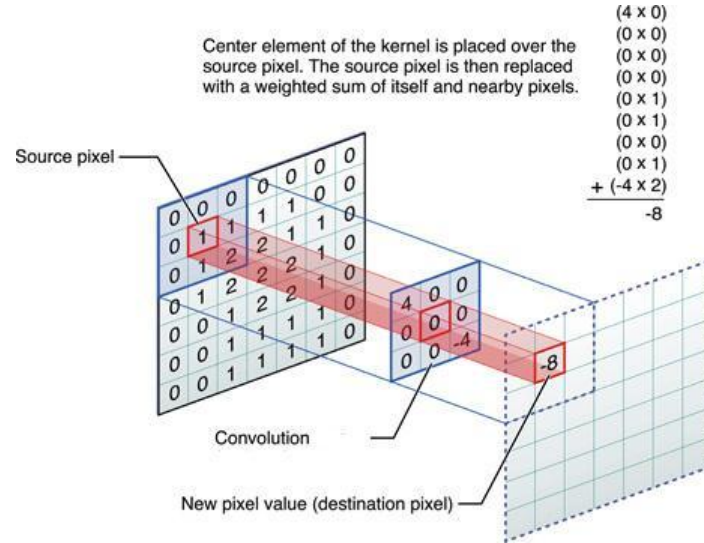
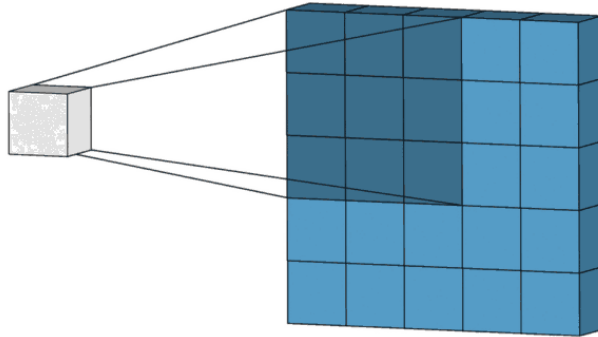
Activation Map

=

Feature Map

¿Qué es una convolución?

- Otra forma de verlo...



Idea clave

¡En ConvNets, estos valores/coeficientes se aprenden! No son seleccionados por un experto humano. Ahora son parámetros libres de la red y se entrenan como cualquier otro peso.



1989: LeCun et al. used **back-propagation to directly learn the coefficients of convolutional filters** from images of handwritten numbers.

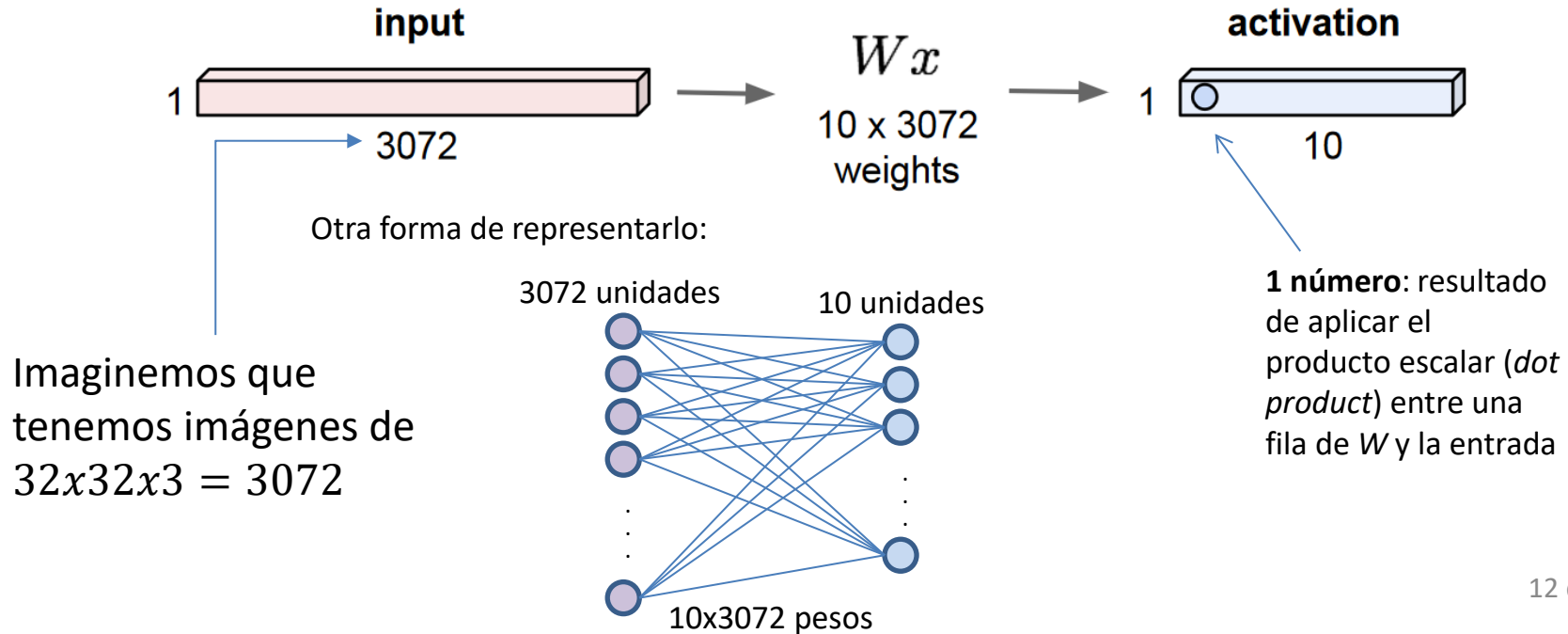
1998: LeCun et al. presented **LeNet-5**, ConvNet with 5 layers that could automatically recognize handwritten numbers, and showed that **ConvNets outperformed all other techniques in this task**.

LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4), 541-551.

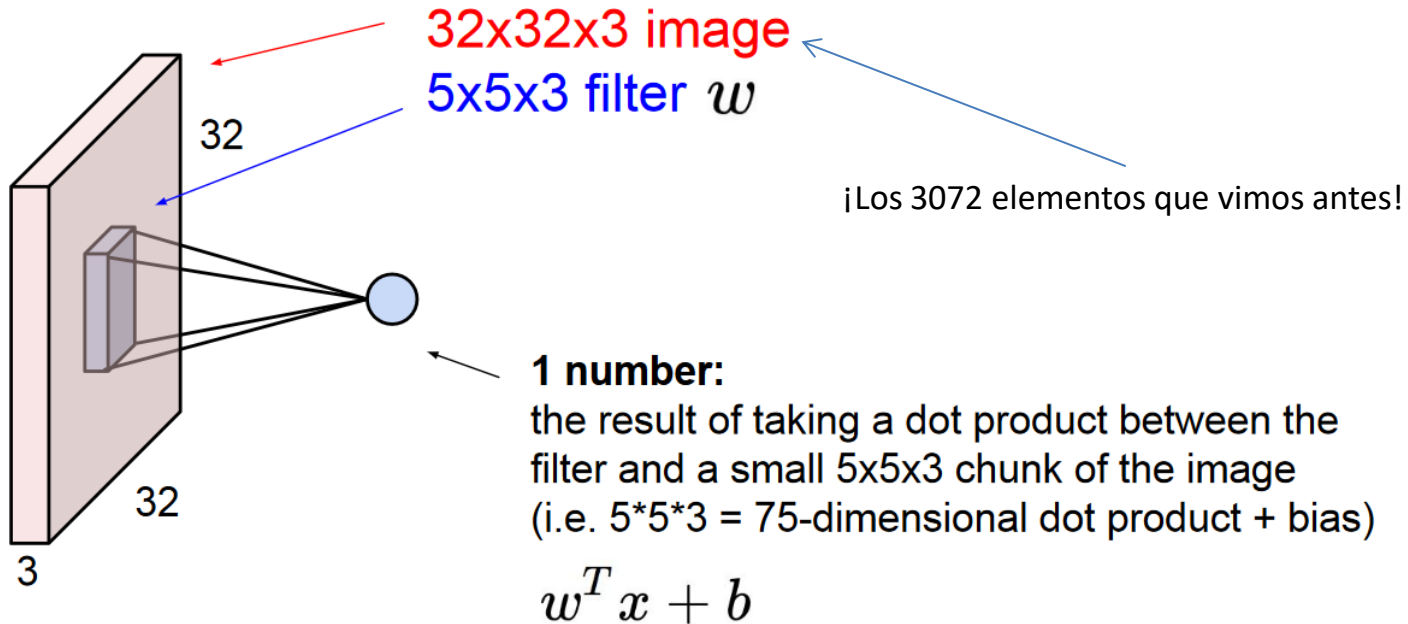
LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

Capa totalmente conectada (*Fully-connected or dense layer*)

Todas las unidades/neuronas de una capa están conectadas (pesos) con todas las unidades/neuronas de la siguiente capa.



Capa Convolutiva

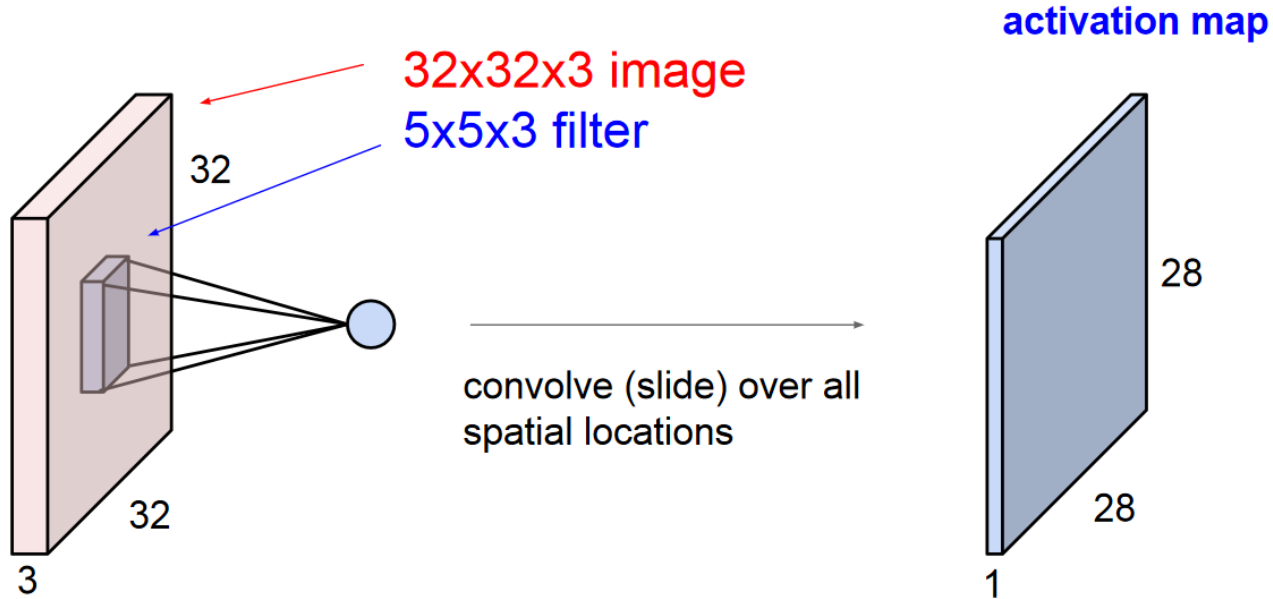


Gran ventaja:
76 pesos vs 30720!

Nota: recordemos que el *bias* aporta flexibilidad al aprendizaje (representaciones más ricas del espacio de entrada), permitiendo adaptar la salida de cada unidad. Ejemplo simple e intuitivo:

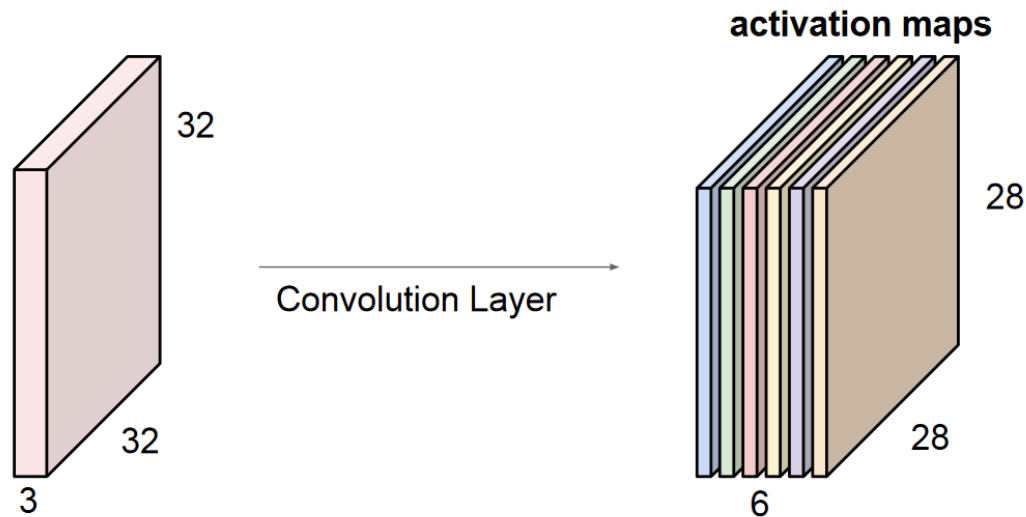
<https://stackoverflow.com/questions/2480650/what-is-the-role-of-the-bias-in-neural-networks>

Capa Convolutacional



Capa Convolutiva

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



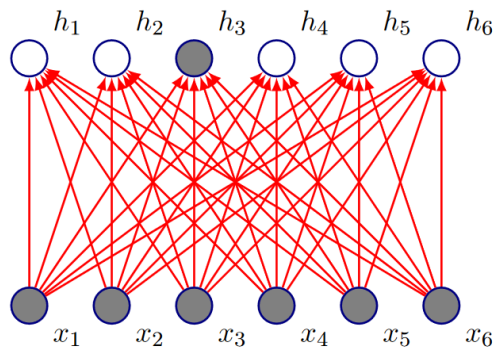
We stack these up to get a “new image” of size 28x28x6!

Incluso ahora el número de pesos es mucho menor que lo que teníamos antes: 456 pesos vs 30720!

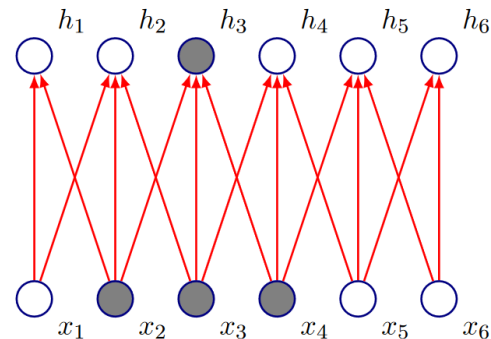
Ventajas de las ConvNets

- **Conectividad dispersa**

- *Fully-connected layers* operan globalmente (cada neurona/unidad “ve” toda la entrada), mientras que las capas convolucionales operan localmente → ¡Menos operaciones!



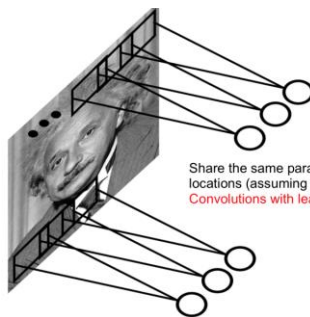
vs



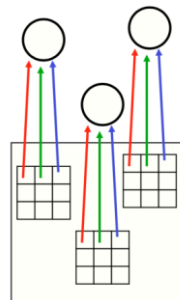
- Fully connected network: h_3 is computed by full matrix multiplication with no sparse connectivity
- Kernel of size 3, moved with stride of 1
- h_3 only depends on x_2, x_3, x_4

Ventajas de las ConvNets

- **Compartición de parámetros (*Weight Sharing*)**
 - El mismo filtro se aplica a toda la imagen.
 - En lugar de aprender un parámetro para cada localización en la imagen, solo se aprende un conjunto reducido de parámetros (los correspondientes al filtro).
 - El número de parámetros a aprender y almacenar se reduce considerablemente → ¡Regularización!



The red connections all have the same weight.



Ventajas de las ConvNets

- **Representaciones equivariantes**

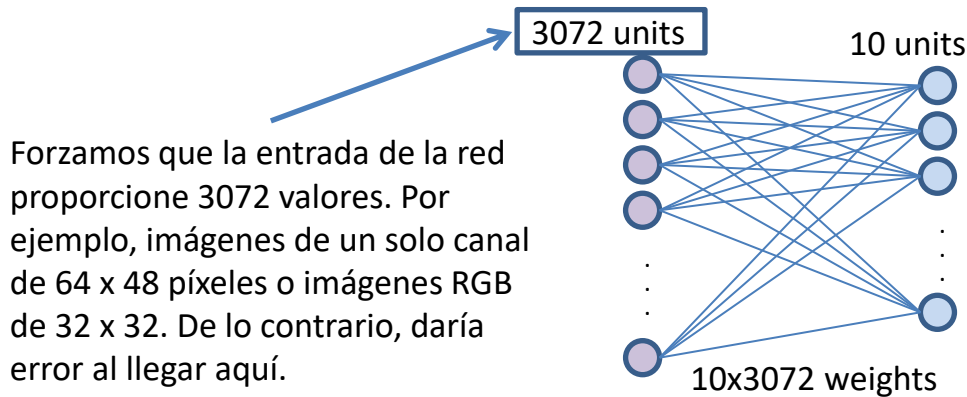
- Toda capa convolucional es equivariante con respecto a la traslación.

$$f \text{ es equivariante con respecto a } g \text{ si } f(g(\mathbf{x})) = g(f(\mathbf{x}))$$

- Si trasladamos un objeto en la imagen, su representación se trasladará a la misma distancia en la salida.
 - Permite generalizar la detección de bordes, texturas y formas en diferentes ubicaciones de la imagen
- La convolución no es equivariante con respecto al escalado o la rotación.

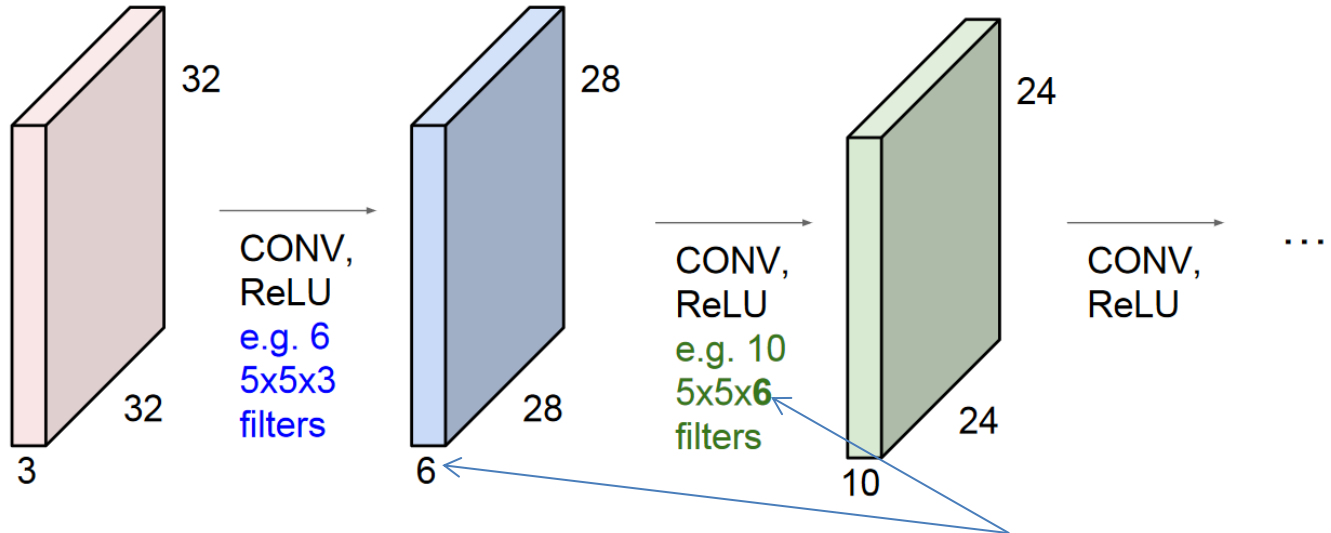
Ventajas de las ConvNets

- (Teóricamente) permiten **operar con entradas de tamaño variable**
 - Una *fully-connected layer* fuerza a que la entrada tenga un determinado tamaño.



Convolutional Networks

- Una ConvNet es una secuencia de capas convolucionales intercaladas con funciones de activación.

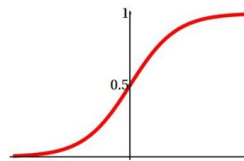


Nota muy importante: los filtros siempre tienen la misma profundidad que el bloque convolucional anterior, es decir, extienden toda la profundidad del volumen/tensor de entrada.

Convolutional Networks

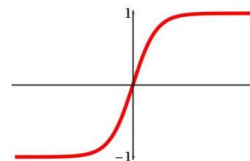
- Las capas convolucionales se intercalan con **funciones de activación (no lineales)**.
 - Tradicionalmente, las funciones de activación más empleadas eran **sigmoides**, como la *logistic sigmoid* o la hiperbólica tangente.
 - En redes profundas generalmente se recomienda utilizar la unidad lineal rectificada (**ReLU**), y sus variantes, entre otras razones porque permite entrenar más rápido.

$$\sigma(\Sigma) = \frac{1}{1 + e^{-\Sigma}}$$

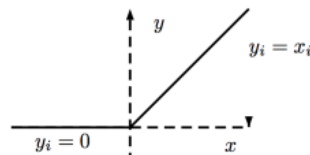


logistic (sigmoid, unipolar)

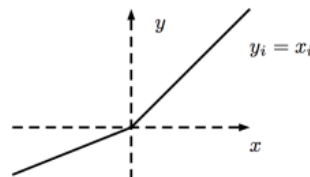
$$\tanh(\Sigma) = \frac{e^{\Sigma} - e^{-\Sigma}}{e^{\Sigma} + e^{-\Sigma}}$$



tanh (bipolar)



ReLU



Leaky ReLU/PReLU

Véase “dying ReLU problema” y “vanishing gradient problem”
([“Yes, you should understand backprop”](#) by Andrej Karpathy)

Convolutional Networks

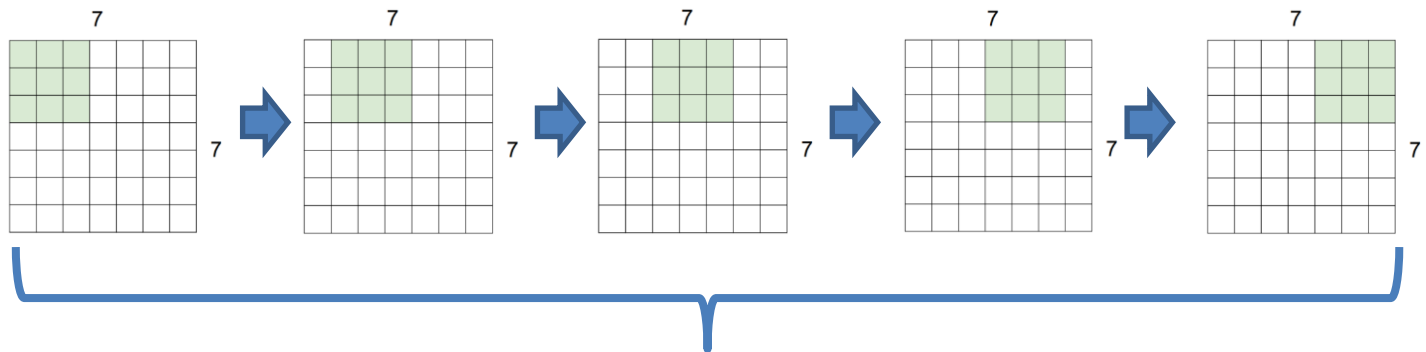
- Las capas convolucionales se intercalan con **funciones de activación (no lineales)**.
 - Esta no-linealidad es **necesaria para aprender representaciones complejas de los datos de entrada**.
 - De lo contrario, si solo se usaran funciones de activación lineal, la red neuronal se comportaría como una función lineal.

If the activation functions of all the hidden units in a network are taken to be linear, then for any such network we can always find an equivalent network without hidden units.

"Pattern Recognition and Machine Learning" (C. M. Bishop, 2016) p.229

Convolutional Networks

- Prestemos atención a las dimensiones:
 - Ejemplo: imagen de 7x7 y filtro de 3x3

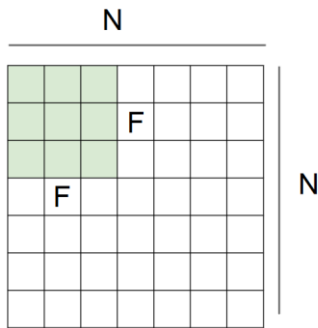
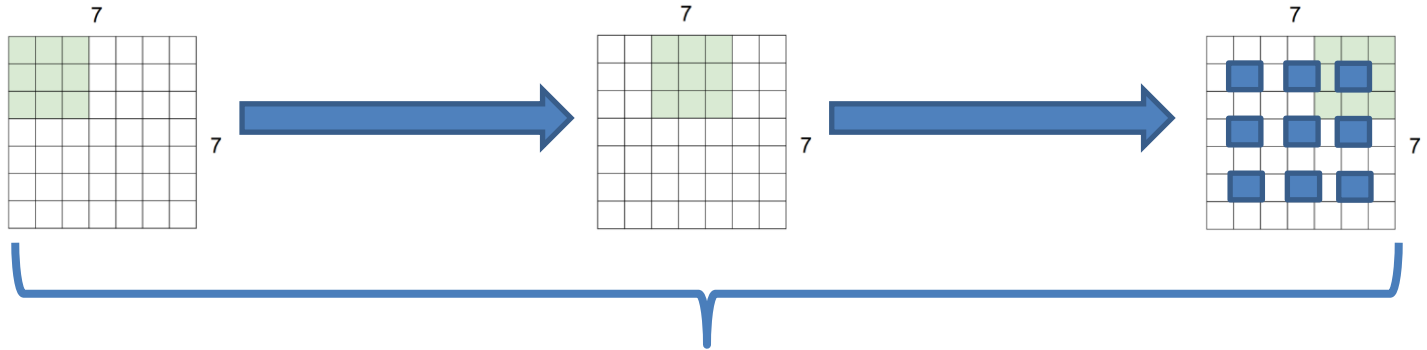


La salida es una imagen de 5x5
(no “reemplazamos” los bordes)

- Conceptos de **stride** y **padding**.

Convolutional Networks

- Prestemos atención a las dimensiones:
 - Ejemplo: imagen de 7x7 y filtro de 3x3, con **stride 2**



La salida es una imagen 3x3

Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7, F = 3$:
stride 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$
stride 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$
stride 3 $\Rightarrow (7 - 3) / 3 + 1 = 2.33$

¡Mal! A priori, aunque siempre hay trucos, no es posible aplicar un filtro 3x3 a una entrada de 7x7 con stride 3!

Convolutional Networks

- Progresivamente reducimos el tamaño de los mapas/volúmenes.
 - Si queremos conservar el tamaño → padding
 - Ejemplo: *zero-padding*

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

En muchos casos es bueno utilizar *padding*, ya que la evidencia empírica aconseja no reducir la dimensionalidad demasiado rápido.

Convolutional Networks

- Ejemplo completo:
 - Volumen de entrada: **16x16x3**
 - Filtros usados: **10** filtros de **3x3** con stride **1** y padding **1**

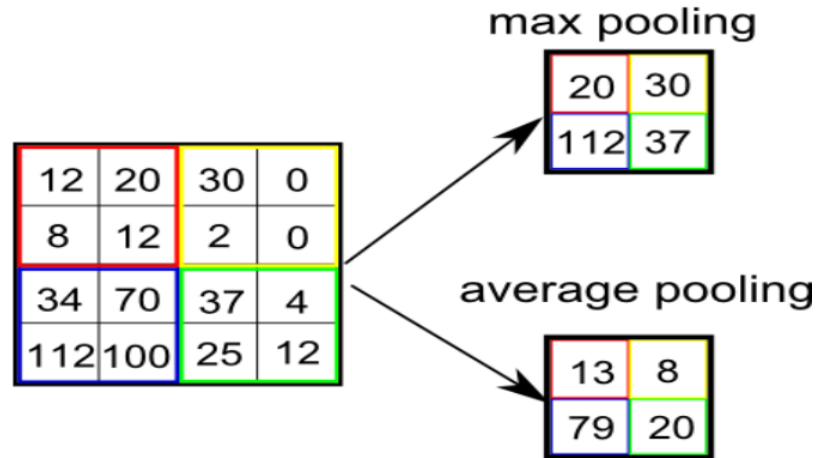
$$\text{Output_size} = ((N + 2 * P - F) / \text{stride}) + 1$$

- Volumen de salida?
 - $((16 + 2 * 1 - 3) / 1) + 1 = 16 \rightarrow 16 \times 16 \times 10$
 - Número de parámetros en esa capa?
 - Cada filtro tiene $3 \times 3 \times 3 + 1 = 28$ parámetros
 $\rightarrow 28 \times 10 = 280$ parámetros
- +1 por el *bias*

Convolutional Networks

- **Pooling.** Reduce dimensionalidad e introduce cierta invarianza a (pequeñas) translaciones en la entrada
 - Si la entrada se desplaza una pequeña distancia, la mayoría de los valores de salida no cambiarían.

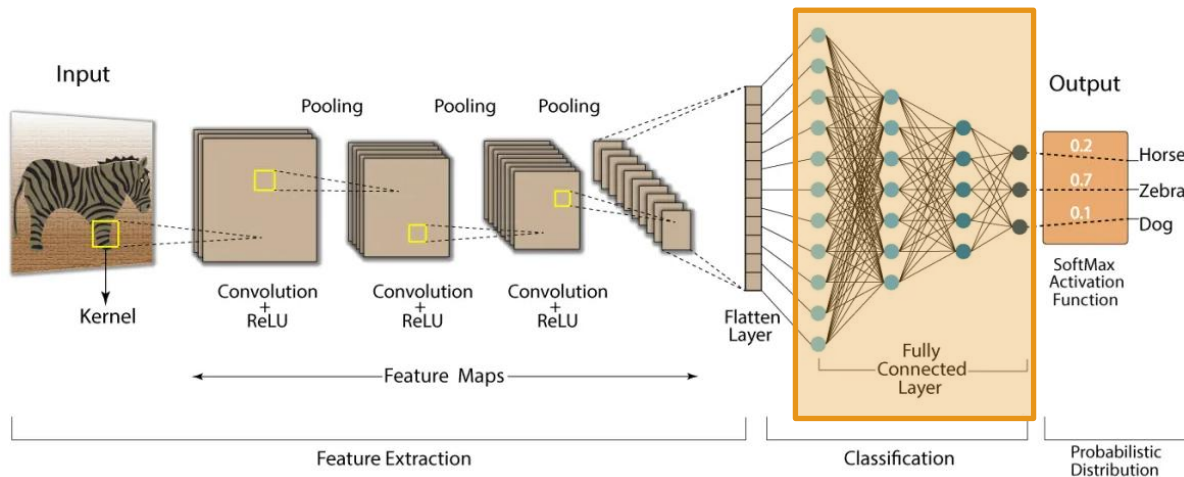
Ejemplo de *max* y *average pooling* de tamaño 2x2 y stride 2



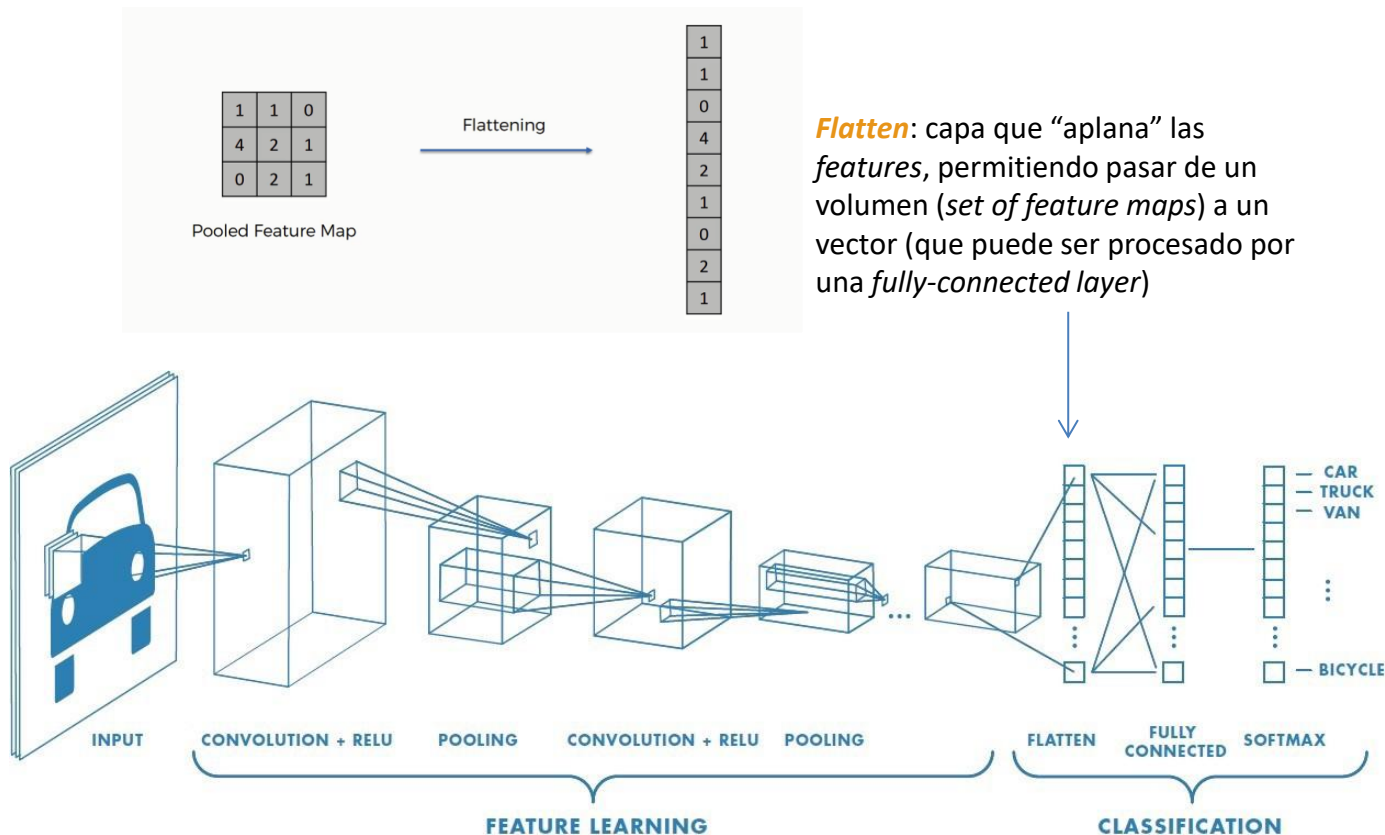
Convolutional Networks

- **Fully-connected** o **dense layers**.

- Las redes convolucionales pueden tener o no capas completamente conectadas.
- Cuando estas están presentes, suelen aparecer al final de la red.
- Todas las unidades de una capa están conectadas a todas las unidades de la siguiente capa.
- Suelen contener muchos parámetros, ¡así que cuidado al incluirlas!



Combinando todo



Combinando todo

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

σ = softmax

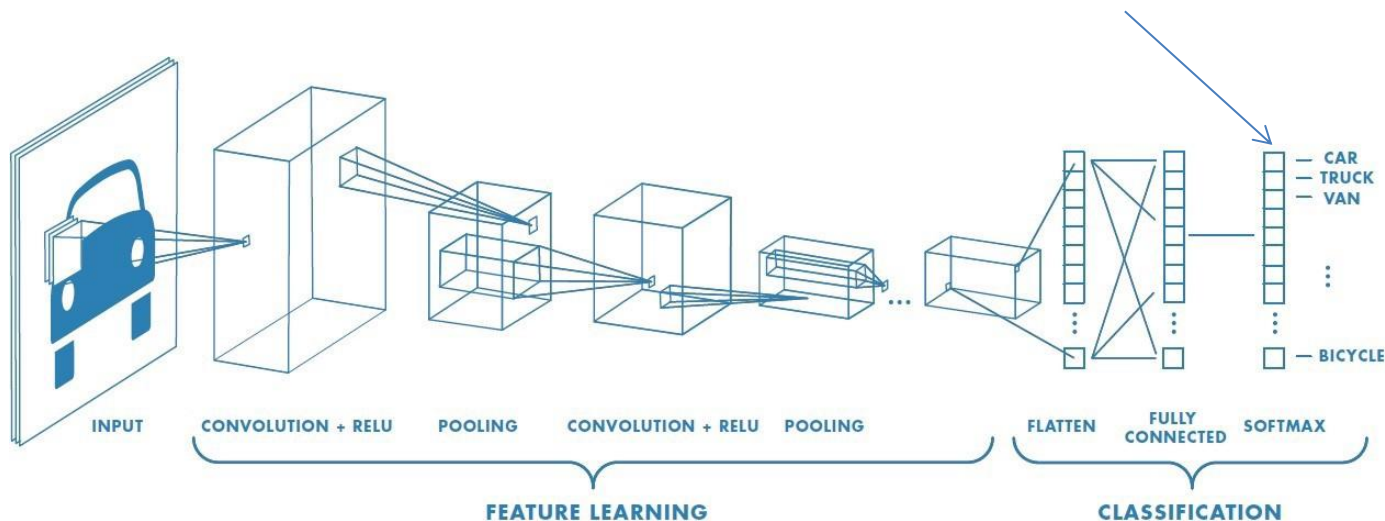
$\vec{z}$ = input vector

e^{z_i} = standard exponential function for input vector

K = number of classes in the multi-class classifier

e^{z_j} = standard exponential function for output vector

Softmax: función de activación que generaliza la función sigmoide a múltiples clases. Normaliza la salida de la red, de modo que cada predicción corresponda a la probabilidad de que la entrada pertenezca a esa clase.

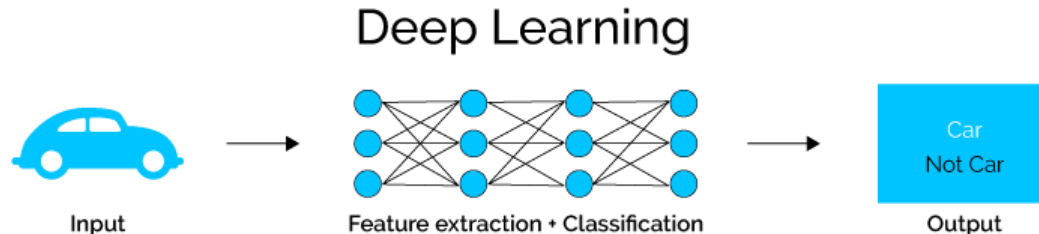
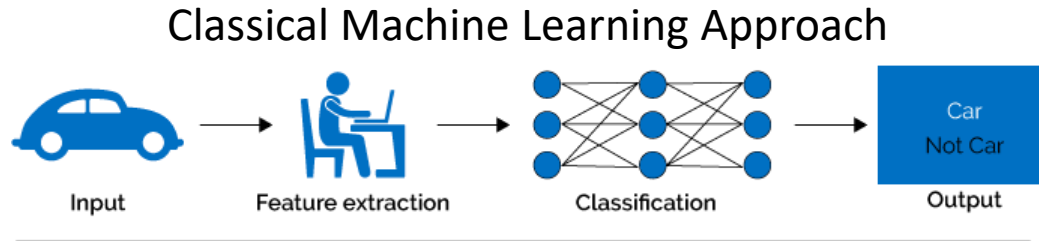


Visualizando modelos

- ConvNetJS. Deep Learning in your browser:
<https://cs.stanford.edu/people/karpathy/convnetjs/>
- A Neural Network Playground: <https://playground.tensorflow.org/>
- An Interactive Node-Link Visualization of Convolutional Neural Networks: https://adamharley.com/nn_vis/
- CNN Explainer: <https://poloclub.github.io/cnn-explainer/>
- ConvNet Playground: <https://convnetplayground.fastforwardlabs.com/>
- Topological visualisation of a convolutional neural network:
<https://terencebroad.com/works/cnn-vis>

Machine Learning vs Deep Learning

- Metodológicamente: ConvNets permiten aprender características (*feature learning*) en lugar de tener que diseñarlas a mano (*feature engineering*).
- Empíricamente: proporcionan **resultados superiores en muchas tareas**.

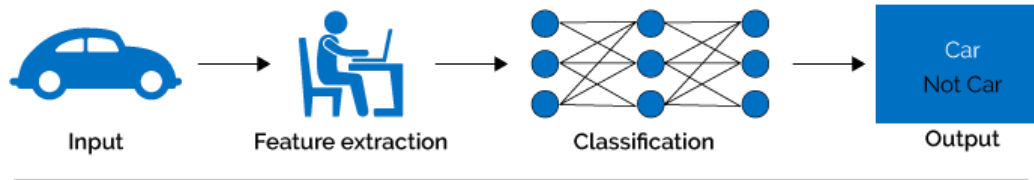


<https://towardsdatascience.com/cnn-application-on-structured-data-automated-feature-extraction-8f2cd28d9a7e>

Machine Learning vs Deep Learning

- Deep Learning difumina los límites entre *feature extraction* e *image classification* (*end-to-end learning*).
- La clave es disponer de una **buena representación interna** de los datos de entrada.

Classical Machine Learning Approach



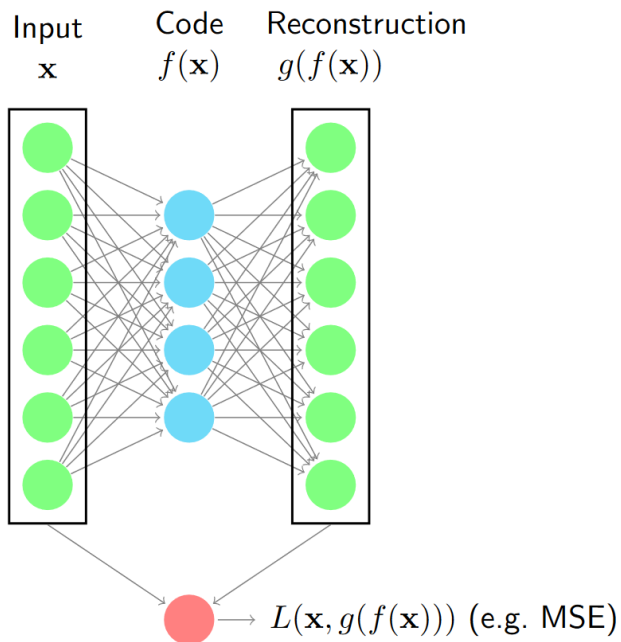
Deep Learning



<https://towardsdatascience.com/cnn-application-on-structured-data-automated-feature-extraction-8f2cd28d9a7e>

Autoencoders

- Arquitecturas *encoder-decoder* que pueden ser empleadas en múltiples tareas, desde compresión a **eliminación de ruido** (*denoising*).

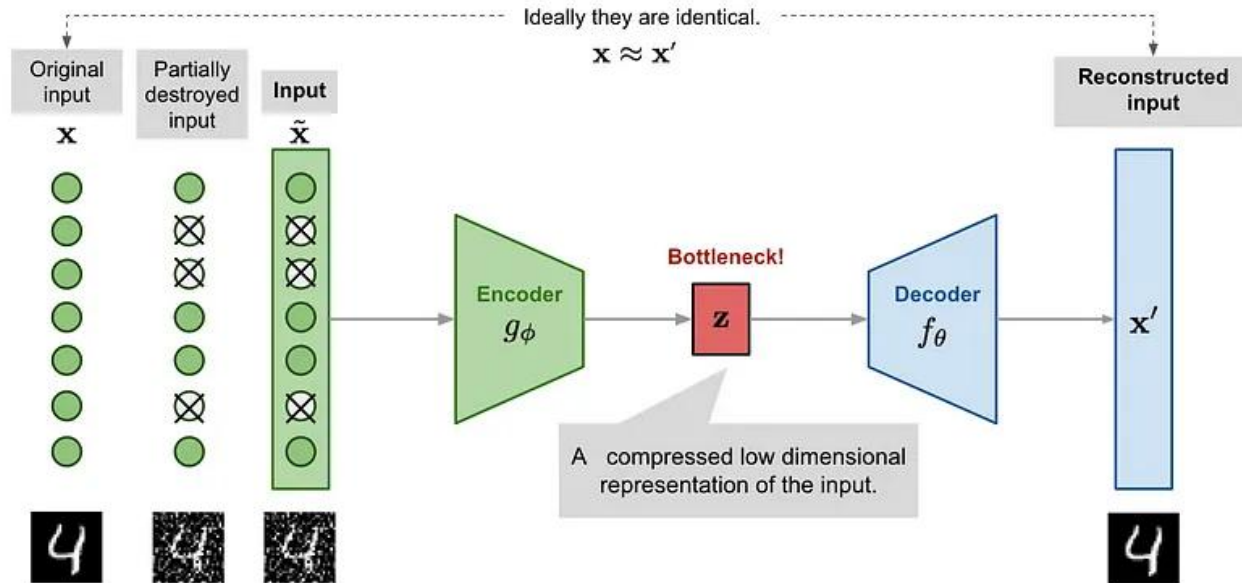


Eliminación de ruido usando autoencoders:

- Corrompemos los datos de entrada a propósito, agregando ruido o enmascarando algunos de los valores de entrada.
- El modelo está entrenado para predecir los datos originales no dañados.
- El autoencoder debe deshacer esta corrupción.

Queremos aprender una forma eficiente de codificar los datos de entrada.

Denoising Autoencoders



[Fuente](#)